Programación Orientada a Objetos

Diseño de Software Orientado a Objetos con Diagramas de Clases

Introducción al Diseño Orientado a Objetos

El **diseño orientado a objetos (DOO)** es el proceso de planificar un sistema de software usando los conceptos de la Programación Orientada a Objetos (POO). Su objetivo es crear un modelo conceptual del sistema antes de escribir el código, mejorando la comprensión, el mantenimiento y la escalabilidad.

¿Por qué usar Diagramas de Clases?

Los **diagramas de clases UML (Unified Modeling Language)** son una notación gráfica estándar que nos permite:

- **Visualizar** la estructura estática de un sistema.
- **Representar** clases, sus atributos, métodos y las relaciones entre ellas.
- **Comunicar** el diseño de forma clara y concisa a otros desarrolladores y interesados.

Introducción al Diseño Orientado a Objetos

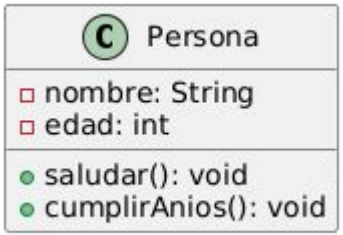
Clases Simples en Diagramas de Clases

Una **clase simple** es la representación más básica de una plantilla para objetos. En un diagrama de clases, se dibuja como un **rectángulo dividido en tres secciones**:

1. **Nombre de la Clase:** Arriba del todo.
2. **Atributos:** Las propiedades o características que describen los objetos de esa clase (ej., nombre, edad, saldo). Se suele indicar su visibilidad (+ público, - privado) y su tipo de dato.
3. **Métodos:** Las operaciones o comportamientos que pueden realizar los objetos de esa clase (ej., saludar(), depositar()). También se indica su visibilidad y, opcionalmente, sus parámetros y tipo de retorno.

Ejemplo: Clase Persona

Representemos una clase Persona con algunos atributos y métodos básicos.



Introducción al Diseño Orientado a Objetos

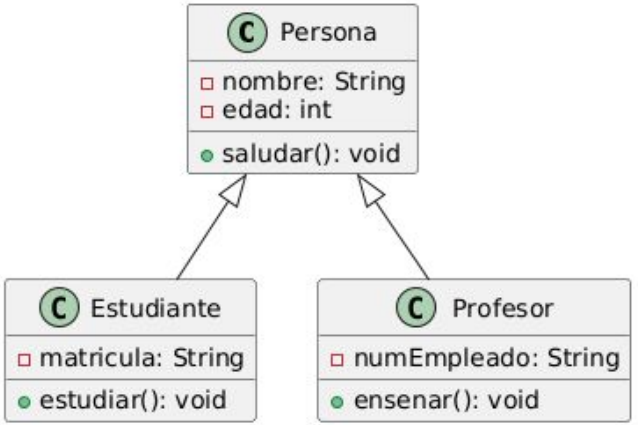
Relación de Herencia

La **herencia** es un pilar de la POO que permite a una clase (**clase hija** o **subclase**) heredar atributos y métodos de otra clase (**clase padre** o **superclase**). Esto fomenta la reutilización de código y establece una relación "es un/a" (ej., "Un Estudiante **es una** Persona").

- En UML, la herencia se representa con una **línea sólida con una flecha hueca y cerrada** apuntando desde la clase hija hacia la clase padre.

Ejemplo: Herencia de Persona a Estudiante y Profesor

Consideremos que un Estudiante y un Profesor son tipos específicos de Persona.



Introducción al Diseño Orientado a Objetos

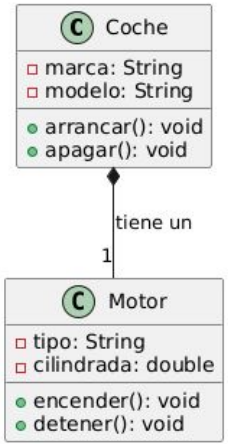
Relación de Composición

La **composición** es un tipo fuerte de relación de "tiene un/a" o "es parte de" entre clases. Implica que un objeto de una clase **contiene** objetos de otra clase, y que la **existencia del objeto contenido depende de la existencia del objeto que lo contiene**. Si el "contenedor" se destruye, el "contenido" también lo hace.

- En UML, la composición se representa con una **línea sólida con un rombo sólido y relleno** en el lado de la clase "contenedora" y una flecha abierta en el lado de la clase "contenida".

Ejemplo: Composición entre Coche y Motor

Un Coche **tiene un** Motor, y el Motor no tiene sentido si el Coche al que pertenece no existe.



- La clase Coche está compuesta por la clase Motor.
- El **rombo sólido** en el lado de Coche indica que Coche es el contenedor (la parte "entera").
- La multiplicidad "1" indica que un Coche tiene exactamente un Motor.
- Esta relación implica que la vida del Motor está ligada a la vida del Coche. Si el Coche es destruido, el Motor asociado también lo es.

Introducción al Diseño Orientado a Objetos

Resumen y Conclusiones

1. Los **diagramas de clases** son esenciales para visualizar y comunicar el diseño de sistemas orientados a objetos.
2. Modelamos **clases simples** con sus atributos y métodos.
3. La **herencia** permite la reutilización y establece relaciones "es un/a" entre clases.
4. La **composición** modela relaciones fuertes de "tiene un/a" donde la vida de las partes está ligada al todo.

Al usar estas herramientas, podemos crear diseños de software más robustos, flexibles y fáciles de entender.